

Lab: Breaking JPEG on Purpose

This is a computer lab for the lecture [5. Lossy Compression: Images and Audio](#). Each JPEG stage exists for a reason. In this lab you change or remove one stage at a time, look at the decoded image, and explain what went wrong.

Files

- `jpeg_lab.py`: a JPEG-like encoder and decoder with command-line options for every stage (Python 3, needs `numpy` and `Pillow`).
- Test images (right-click and “Save link as...”):
 - `bumblebee.png`: a photo, 400×300 pixels.
 - `kuldiga1.png`: a photo, 600×450 pixels.
 - `mandelbrot-picture.png`: a computer-generated image with saturated colors and sharp edges, 850×850 pixels.
 - `test-chart.png`: a synthetic test chart with color bars, colored text, thin lines and a checkerboard, 512×384 pixels. It was drawn by `make_test_chart.py`.
- You can also use your own images. Small images (up to about 1000×1000 pixels) are easier to compare side by side and are processed in about a second.

```
pip install numpy pillow
python jpeg_lab.py bumblebee.png out.png
python jpeg_lab.py --help
```

PNG is only used to store the RGB pixels. `jpeg_lab.py` does not create a `.jpg` file. It runs the JPEG stages on the pixels, decodes the result immediately and saves the decoded pixels as a PNG image. It also prints:

- the error for each color channel: the mean squared error (MSE) and $\text{PSNR} = 10 \log_{10} \frac{255^2}{\text{MSE}}$ in decibels (more is better; above about 40 dB the error is hard to see);
- the number of nonzero quantized coefficients;
- the estimated compressed size. The coefficients are converted to the baseline JPEG symbols (DC differences, (RUN, SIZE) pairs, ZRL, EOB), and the entropy of the symbols is added to the number of extra bits. With default settings the estimate is within about 5% of the size of a real JPEG file of the same quality that uses optimized Huffman tables (headers not counted). With the standard Huffman tables of Annex K, real files are larger, especially for synthetic images such as `test-chart.png`.

Two extra outputs help to see what happened:

- `--difference-png diff.png` writes the error image: gray (128) means no error, lighter and darker pixels mean that the decoded value is too large or too small (the error is multiplied by `--difference-amplification`, 4 by default).
- `--planes-png planes.png` writes the three color planes (for example Y, Cb, Cr) as gray images: the original planes in the top row and the decoded planes in the bottom row.

Always compare a modified run with the default run (`python jpeg_lab.py image.png default.png`) and, if possible, at the same estimated size. It is easy to get a better image by spending more bits.

How the program is organized

The source code follows the JPEG stages from the lecture. To change a stage, find its section in the file; every option below is stored in `settings.<name>` (for example `--block-size` is `settings.block_size`).

Stage	Functions	Options
Read pixels	load_rgb_pixels	input_png
1. Color space conversion	rgb_to_planes, planes_to_rgb	--color-space {ycbcr,yiq,rgb}
2. Chroma subsampling	subsample,upsample	--chroma-subsampling {4:4:4,4:2:2,4:2:0,4:1:1}
3. Blocks and level shift	pad_to_multiple, split_into_blocks, merge_blocks	--block-size, --no-level-shift
4. Transform	dct_matrix,dst7_matrix, hartley_matrix, walsh_hadamard_matrix, haar_matrix,klt_matrix	--transform
4b. Coefficient selection (not in JPEG)	drop_high_frequencies, scale_up_kept_coefficients, fill_dropped_with_noise	--keep-coefficients, --energy-compensation {none,scale,noise}
5. Quantization	make_quantization_tables, scale_table_by_quality, quantize,dequantize	--quality,--quant-table, --flat-step, --luma-quant-multiplier, --chroma-quant-multiplier,--rounding
6.-7. Zig-zag, run-length, size	zigzag_order, estimate_plane_bits	none
Decoder: deblocking (not in JPEG)	deblock	--deblocking-filter, --deblocking-threshold

Some details:

- Every transform is an orthonormal $N \times N$ matrix T . A block A is transformed to $B = TAT^T$ and restored as $A = T^TBT$. For the DCT, T is the matrix C from the lecture.
- For block sizes other than 8, the 8×8 quantization tables are stretched: the coefficient (u, v) of an $N \times N$ block gets the entry $Q_{\lfloor 8u/N \rfloor, \lfloor 8v/N \rfloor}$ of the table.
- `--quality` scales the tables as *libjpeg* does (see step 5 in the lecture). Then `--luma-quant-multiplier` multiplies the table of the first plane, and `--chroma-quant-multiplier` multiplies the table of the second and the third plane.
- The decoder upsamples chroma by repeating samples. Real decoders often interpolate, so their images look slightly smoother.

Exercise 1: No color space conversion

With `--color-space rgb` the program skips stage 1: R is coded as if it were Y , G as Cb and B as Cr . So G and B are subsampled and quantized with the chroma table, which is coarser.

```
python jpeg_lab.py bumblebee.png ybcbcr.png
```

```
python jpeg_lab.py bumblebee.png rgb.png --color-space rgb
python jpeg_lab.py test-chart.png chart-ycbcr.png --planes-png
↪ chart-ycbcr-planes.png
python jpeg_lab.py test-chart.png chart-rgb.png --color-space rgb
```

1. Compare the images, the PSNR and the estimated size. Which picture is more blurred, and why? Recall which of R , G , B has the largest weight in $Y = 0.299R + 0.587G + 0.114B$.
2. Look at the vertical black lines on `test-chart.png` in both results. Where does the color that appears in `chart-rgb.png` come from?
3. Now switch off subsampling and use the same table for all three planes, so that only the color space differs:

```
python jpeg_lab.py test-chart.png a.png --chroma-subsampling 4:4:4
↪ --quant-table luma-for-all
python jpeg_lab.py test-chart.png b.png --chroma-subsampling 4:4:4
↪ --quant-table luma-for-all --color-space rgb
```

Which one is smaller? Look at `--planes-png` for both runs. Why do the Cb and Cr planes of a typical photo compress so well, while the G and B planes do not?

4. Quantize the second and third plane more aggressively. Double the chroma table (`--chroma-quant-multiplier 2`), then try 4 and 8. Do it in both color spaces (with `--chroma-subsampling 4:4:4`):

```
python jpeg_lab.py bumblebee.png c.png --chroma-subsampling 4:4:4
↪ --chroma-quant-multiplier 4
python jpeg_lab.py bumblebee.png d.png --chroma-subsampling 4:4:4
↪ --chroma-quant-multiplier 4 --color-space rgb
```

In which color space can the chroma planes be quantized coarsely without visible damage? What kind of error appears in RGB: wrong brightness or wrong color?

5. Repeat step 1 with `--color-space yiq`. Is there a noticeable difference between YIQ and YCbCr? Why (see the lecture: how are the IQ and UV planes related)?

Exercise 2: Large blocks and blocking artifacts

```
python jpeg_lab.py bumblebee.png b8.png --quality 10
python jpeg_lab.py bumblebee.png b32.png --quality 10 --block-size 32
python jpeg_lab.py bumblebee.png b32-deblocked.png --quality 10 --block-size 32
↪ --deblocking-filter
```

1. At low quality every block keeps only a few coefficients. The same holds for the average brightness and a gentle slope inside a block. Why do these not match at the borders of neighboring blocks? Why is the grid much easier to see with 32×32 blocks than with 8×8 blocks? Also try `--quality 5`.
2. Find a sharp edge (a leaf or a flower petal) and look at the ringing next to it. How far does the ringing spread from the edge with 8×8 blocks and with 32×32 blocks? What does this tell you about the choice of block size (compare with the adaptive block sizes of AVIF in the lecture)?
3. The deblocking filter (function `deblock`) looks at three pixels on each side of a block border. If the step across the border is smaller than `--deblocking-threshold` and both sides are smooth, the step is replaced by a linear ramp. Try thresholds 10, 24, 60, 200. What happens to real edges of the picture when the threshold is too large? Why can the filter not remove the grid completely?

4. Errors can also be concentrated *next to* the block borders because of the transform. Compare the error images:

```
python jpeg_lab.py bumblebee.png dct16.png      --block-size 16 --difference-png
↳ dct16-diff.png
python jpeg_lab.py bumblebee.png hartley16.png --block-size 16 --transform
↳ hartley --difference-png hartley16-diff.png
python jpeg_lab.py bumblebee.png dst16.png      --block-size 16 --transform dst7
↳ --difference-png dst16-diff.png
```

The Hartley transform (a real-valued version of the discrete Fourier transform) treats a block as one period of a periodic signal, so the left edge of the block “continues” into its right edge. The DCT behaves as if the block were extended by its mirror image. The DST-VII assumes that the signal is 0 just before the top/left edge. For each transform, draw a 1-D row of a block with its assumed extension and explain where the extension has a jump. Where do the largest errors appear in the error images?

Exercise 3: Keeping only the low frequencies

`--keep-coefficients K` keeps only the top-left $K \times K$ corner of each block (the DC coefficient and the lowest frequencies) and replaces the other coefficients with 0. Use a high quality setting, so that the dropping and not the quantization causes the error.

```
python jpeg_lab.py bumblebee.png keep3.png --keep-coefficients 3 --quality 90
python jpeg_lab.py test-chart.png chart-keep3.png --keep-coefficients 3 --quality
↳ 90
```

1. Of 64 coefficients, 9 remain. What is the resolution of the result in terms of “details per block”? Which parts of the test chart are destroyed completely, and which survive? Try $K = 1, 2, 4, 6$.
2. By Parseval’s identity (see the lecture) the squared error of a block is exactly the sum of the squares of the dropped coefficients. For $K = 3$, compare the PSNR with that of the default run at the same estimated size (adjust `--quality` of the default run). Which image looks better? What does this say about dropping coefficients by position compared to quantizing them by size?
3. Now compensate the lost energy. With `--energy-compensation scale`, the encoder multiplies the kept AC coefficients of each block by a factor such that the energy (sum of squares) of the block stays the same. With `--energy-compensation noise`, the decoder fills the dropped positions with random numbers that have the same total energy (“noise filling”; the encoder sends one number per block, the dropped energy).

```
python jpeg_lab.py bumblebee.png keep3-scale.png --keep-coefficients 3
↳ --quality 90 --energy-compensation scale
python jpeg_lab.py bumblebee.png keep3-noise.png --keep-coefficients 3
↳ --quality 90 --energy-compensation noise
python jpeg_lab.py test-chart.png chart-keep3-noise.png --keep-coefficients 3
↳ --quality 90 --energy-compensation noise
```

Both images have the correct energy, but the PSNR gets *worse*. Why? Does either image look sharper? Where does the noise help (grass, leaves) and where does it hurt (flat areas, lines, text)?

4. Audio codecs (AAC, Opus) use noise filling for high frequencies, but image codecs almost never use it for edges. AV1 and AVIF only resynthesize film grain. Explain why this idea works better for hiss and grain than for lines and text.

Exercise 4: Why these details? Rounding, table and level shift

Each part changes one small detail of stages 3-5.

(a) Rounding. JPEG rounds $B_{u,v}/Q_{u,v}$ to the *nearest* integer.

```
python jpeg_lab.py bumblebee.png floor.png --rounding floor
python jpeg_lab.py bumblebee.png trunc.png --rounding toward-zero
```

1. With `floor` the image is destroyed and the size grows, although only the rounding has changed. Explain what happens to a small negative coefficient such as $B/Q = -0.1$. How many nonzero coefficients are there compared to the default?
2. With `toward-zero` (dropping the fractional part) the image is only slightly worse, but the size is smaller. Compare it with the default at the *same* size (lower `--quality` of the default run). Real encoders sometimes round small values toward zero on purpose (a *dead zone*) or choose the rounding direction per coefficient (*trellis quantization* in AVIF). Why can this be a good trade-off?

(b) The shape of the quantization table. The standard table has small steps for low frequencies and large steps for high frequencies.

```
python jpeg_lab.py bumblebee.png flat.png --quant-table flat --flat-step 30
```

Find `--flat-step` and `--quality` values so that a run with the flat table and a run with the standard table have the same estimated size. Compare the PSNR and compare the images. Does the higher PSNR also look better? Which kind of error does the eye notice first: noise in fine texture, or errors in large smooth areas?

(c) Level shift. JPEG subtracts 128 from every sample before the DCT.

```
python jpeg_lab.py bumblebee.png no-shift.png --no-level-shift
```

The result is almost identical. Show that for an orthonormal transform whose first basis function is constant, subtracting 128 changes only the DC coefficient (by how much, for $N = 8$?). Why does the standard do it anyway? Hint: the range of $B_{0,0}$, and the fact that DC values are coded as differences.

(d) No transform at all. `--transform identity` quantizes the pixels themselves (the “coefficients” are the pixel values).

```
python jpeg_lab.py bumblebee.png identity.png --transform identity
python jpeg_lab.py bumblebee.png identity-flat.png --transform identity
↪ --quant-table flat --flat-step 40
```

Explain the pattern in the first image (which table entry is used for which pixel?). In the second image each pixel is simply rounded to a multiple of 40. Why is it larger *and* worse than the default run? What does the transform do that quantizing pixels cannot?

Exercise 5: Other orthogonal transforms

Replace the DCT with other orthonormal bases. All of them are used in practice:

<code>--transform</code>	Basis functions	Where it is used
<code>dct</code>	cosines (DCT-II)	JPEG, MPEG-2, H.264/H.265 (integer approximations), AV1

<code>--transform</code>	Basis functions	Where it is used
<code>dst7</code>	sines, 0 at the top/left edge (DST-VII)	H.265 4×4 intra luma blocks, AV1 (ADST)
<code>hartley</code>	$\cos + \sin$ (a real-valued DFT)	spectral analysis; close relative of the FFT
<code>walsh-hadamard</code>	only $+1$ and -1 (sorted by number of sign changes)	H.264 (DC coefficients), video encoders for cost estimates (SATD), CDMA codes
<code>haar</code>	short steps at different scales (simplest wavelet)	wavelet image coding (JPEG 2000 uses smoother wavelets on the whole image)
<code>klt</code>	eigenvectors of the pixel correlation matrix of <i>this</i> image (PCA)	theoretical optimum; face recognition (“eigenfaces”), data analysis
<code>identity</code>	single pixels	no transform (see 4d)

```
python jpeg_lab.py bumblebee.png t.png --transform walsh-hadamard --print-tables
↪ --difference-png t-diff.png
```

1. For each transform, write down the PSNR, the number of nonzero coefficients and the size. Put them in a table and sort by size. Use at least two images (a photo and `test-chart.png`).
2. Look at the images from `walsh-hadamard` and `haar`. What is the typical shape of the artifacts, and how does it follow from the shape of the basis functions (use `--print-tables` to see the matrix rows)?
3. `klt` computes the best orthonormal transform for rows and columns of the given image. Compare its printed matrix (`--block-size 4 --transform klt --print-tables`) with the DCT matrix for $N = 4$ from the lecture. What do you notice? Why does this explain why JPEG uses a fixed DCT and does not send an image-specific KLT matrix?
4. The quantization table was designed for DCT coefficients. Is the comparison fair to the other transforms? Repeat the comparison with `--quant-table flat` at equal size.
5. (Optional, programming.) Add a transform of your own to `transform_matrix` (for example DCT-IV, or a DCT rounded to integers as in H.264, then normalized). Check that $T @ T.T$ is the identity matrix.

Exercise 6 (optional): Your own modification

Change the code in one place, and predict the effect before running it. Some ideas:

- Transmit the *zig-zag* order in reverse, or use a row-by-row order. `estimate_plane_bits` uses `zigzag_order`. Does the estimated size change? Why?
- In `quantize`, add random noise before rounding (*dithering*). Does it help against the grid of exercise 2?
- Upsample chroma with linear interpolation instead of repetition (function `upsample`).
- Quantize only every second block coarsely. Can you see the checkerboard?

Summarize each experiment in two or three sentences: what you changed, what you expected, what you saw, and which stage of real JPEG avoids the problem.